A Comparative Study of Binary Sentiment Analysis on IMDB and SST-2

Cuclea Luca-Nicolae - 341

Rizea Mihai-Marius - 341

Dilmac Cemil-Andrei - 331

Popa Petru - 341

Abstract

Binary sentiment analysis is a fundamental natural language processing (NLP) task that requires models to accurately interpret polarity in text. However, human sentiment is frequently expressed through complex linguistic phenomena, such as irony, negation, and contrastive shifts, which challenge traditional lexical models. This paper presents a comprehensive benchmarking study evaluating an evolutionary spectrum of machine learning architectures on two complementary datasets: SST-2 (short, sentence-level text) and IMDB (long, document-level reviews). Our methodology traces the progression of NLP techniques, beginning with classical statistical baselines (Linear Regression, Support Vector Machines) and hybrid approaches (NB-SVM), advancing to recurrent deep learning architectures (LSTMs), and culminating with state-of-the-art Transformer models (BERT-based). By analyzing performance and error patterns across these paradigms, we demonstrate how transitioning from frequency-based representations (like TF-IDF) to deep contextualized embeddings resolves critical limitations in capturing compositional semantics, long-range dependencies, and contextual nuance.

1 Introduction

For our NLP course project, we decided to tackle binary sentiment analysis, specifically, the classic problem of figuring out whether a movie review is positive or negative. On the surface, this sounds like a pretty straightforward problem to solve. You just look for words like "good" or "terrible," right? But as we quickly realized, human language is incredibly messy. People use irony, complex negation, and contrastive shifts that completely throw off basic models. In our opinion, this makes sentiment analysis one of the most interesting challenges in NLP, because it forces a model to actually "understand" context rather than just functioning as a glorified word-counter.

We chose this topic because we thought it would be interesting to see exactly how different generations and kinds of machine learning models handle this linguistic messiness. To make it a fair and comprehensive test, we are benchmarking our models on two widely used, complementary datasets: the Stanford Sentiment Treebank (SST-2) and the Large Movie Review Dataset (IMDB). We picked these because they offer two completely different environments for our models.

Instead of just jumping straight to the newest technology, we structured our project as an evolving narrative. We want to trace how NLP architectures have progressed over time to solve the specific limitations of their predecessors:

Classical Baselines: We started with traditional models like Linear Regression and Support Vector Machines (SVM) using TF-IDF. They actually perform reasonably well, but our error analysis showed that because they treat features independently, they completely fail to capture word dependencies, word order, or compositional meaning.

Hybrid Models: Next, we implemented a Naive Bayes-SVM (NB-SVM) hybrid to see if combining generative and discriminative linear approaches could give us a stronger, but still fast, baseline.

Deep Learning: To try to fix the "word order" problem, we moved on to Long Short-Term Memory (LSTM) networks. Since LSTMs can capture sequential dependencies, we want to see how well they aggregate sentiment across an entire sentence.

Transformer Architectures: Finally, we brought in BERT-based models. By leveraging deep bidirectional attention, we expect these models to finally understand the complex phrasing, irony, and context that broke our earlier attempts.

By tracking not just the overall accuracy, but the specific kinds of mistakes each model makes across both datasets, we showcase exactly why the NLP field had to evolve beyond simple bag-of-words approaches to truly understand human sentiment.

2 Related Work

Sentiment analysis is usually treated as a text classification problem, where the goal is to decide whether a text expresses a positive or negative opinion. One of the first important works in this area was introduced by Pang et al., who applied machine learning methods such as Naive Bayes and Support Vector Machines to classify movie reviews as positive or negative (Pang et al., 2002). Turney also studied the problem using an unsupervised method, showing that the sentiment of a review can be estimated from the words and phrases used in the text (Turney, 2002). Later, the field became easier to compare because of standard datasets such as IMDB, introduced by Maas et al. for binary movie review classification (Maas et al., 2011), and the Stanford Sentiment Treebank, from which SST-2 is derived for sentence-level sentiment classification (Socher et al., 2013).

Early sentiment analysis systems used simple preprocessing and feature extraction before applying a classifier. The text was usually split into words, cleaned, and transformed into bag-of-words, n-gram or TF-IDF vectors. In this setting, the model mainly learns which words or word groups are more common in positive or negative examples. Naive Bayes was often used because it is simple and fast for sparse text representations, while SVM became one of the strongest traditional baselines because it works well with large text feature spaces (McCallum and Nigam, 1998; Joachims, 1998). Logistic Regression was also commonly used as a simple baseline because it learns weights for the extracted features and gives an interpretable linear decision. A stronger classical method is NBSVM, which combines Naive Bayes log-count ratios with a linear SVM, giving more importance to words that are strongly associated with one class (Wang and Manning, 2012). These models are efficient and easy to compare, but they mostly depend on word counts and have difficulties with word order, negation and longer context.

Neural models changed sentiment analysis by replacing manually designed sparse features with learned representations. Word embeddings such as Word2Vec and GloVe represent words as dense vectors, so words with similar meanings can have similar representations (Mikolov et al., 2013; Pennington et al., 2014). CNN models use these embeddings and apply filters over short groups of words, which helps them detect local expressions that are important for sentiment, such as short positive or negative phrases (Kim, 2014). LSTM models process the review as a sequence, token by token, and keep an internal state that carries information from previous words (Hochreiter and Schmidhuber, 1997). This makes them more suitable than classical models for cases where the meaning depends on word order, negation or contrast. For longer texts such as IMDB reviews, hierarchical attention models were proposed to first represent words inside sentences and then combine sentence information into a document representation (Yang et al., 2016).

Transformer models introduced a different way of processing text. Instead of reading tokens one after another like an LSTM, they use attention to compare different parts of the same text at the same time (Vaswani et al., 2017). This is useful in sentiment analysis because the meaning of a word can depend on another word that appears earlier or later in the sentence. BERT uses this Transformer architecture and is first trained on large collections of text, then adapted to a specific task such as sentiment classification (Devlin et al., 2019). Before the text is passed to the model, BERT uses a tokenizer that can split uncommon words into smaller parts, which helps it deal with words that may not appear often in the training data. For IMDB or SST-2, BERT is then fine-tuned on labeled examples so that the model learns to classify each review or sentence as positive or negative.

RoBERTa is based on the same general idea as BERT, but improves the way the model is trained before fine-tuning. Liu et al. showed that BERT can be improved by training it longer, on more data, with better training settings, and by removing one of the original training objectives that was not very useful (Liu et al., 2019). In practice, RoBERTa is used for sentiment analysis in almost the same way as BERT: the text is tokenized, passed through the pretrained model, and then the model is fine-tuned on labeled positive and negative examples. The difference is that RoBERTa usually starts from a stronger pretrained model, which often leads to better results on benchmarks such as SST-2.

Recent work has also used supervised contrastive learning to improve the representations learned during fine-tuning. The idea is to make examples with the same label closer to each other and examples with different labels farther apart (Khosla et al., 2020). Gunel et al. applied this idea to pretrained language model fine-tuning, making it relevant for

models such as RoBERTa combined with supervised contrastive learning (Gunel et al., 2021). In binary sentiment analysis, this can help the model learn a clearer separation between positive and negative reviews, instead of only learning to predict the correct label through the standard classification loss. Although there is no single universal state-of-the-art model for every sentiment analysis setting, current high-performing approaches on benchmarks such as IMDB and SST-2 are generally based on pretrained Transformer models fine-tuned for classification. Therefore, comparing Logistic Regression, Naive Bayes, SVM, NBSVM, LSTM, BERT, RoBERTa and RoBERTa with supervised contrastive learning on IMDB and SST-2 gives a clear view of how binary sentiment analysis evolved from simple word-count models to modern Transformer-based models.

3 Method

Binary sentiment classification is a well-studied problem, but performance can change a lot depending on the text's length and complexity. To evaluate this across two different benchmarks — SST-2 (short, sentence-level reviews) and IMDB (long, full-length reviews) — we set up an experiment using both traditional Machine Learning baselines and modern Deep Learning architectures. This allows us to clearly measure the trade-offs between a model's complexity, its speed, and its accuracy.

To fully evaluate sentiment classification across varying text lengths and complexities, we approach the problem from two different angles. First, we establish fast, transparent baselines using traditional Machine Learning algorithms, which are easy to interpret and require very little computational power. Second, we use State-of-the-Art Deep Learning architectures capable of capturing deep contextual meaning — addressing issues like long-distance negations and sarcasm that usually confuse classical methods. This dual strategy allows for a fair comparison between lightweight, explainable models and high-capacity neural networks across both datasets. The following subsections explain the setup and analysis strategy applied to these models.

Traditional Machine Learning Baselines

Our classical baselines use TF-IDF vectorization to turn raw text into numerical features. Despite their simplicity, these models are essential starting points: they establish a minimum performance threshold, are fast to train, and offer enough transparency to easily see how words influence predictions. We evaluate three models:

- **Logistic Regression:** A fast, standard probabilistic baseline that predicts classes using a linear combination of TF-IDF features. It provides a reliable starting score and makes it easy to understand the weight of each word.
- **Linear SVM:** Chosen for its proven ability to perform well in high-dimensional spaces — exactly the setting produced by our TF-IDF matrices, which contain up to 200,000 features. Its margin-maximizing approach helps it handle this kind of data effectively.
- **NB-SVM (Naive Bayes – Support Vector Machine):** A hybrid architecture that applies a Naive Bayes log-count ratio transformation,

$$r = \log\left(\frac{p/\|p\|_1}{q/\|q\|_1}\right),$$

  to re-weight the TF-IDF features before classification. This gives the SVM stronger sentiment signals and helps it deal better with overlapping single words.

State-of-the-Art Deep Learning Architectures

To address the limitations of classical models, we use two Transformer-based architectures that rely on deep context learned from large-scale pretraining. Unlike bag-of-words methods, these models can link words that are far apart in a sentence. This enables them to correctly interpret complex language — such as negations and sarcasm — that usually confuse simple n-gram approaches:

- **BERT:** Our main Deep Learning baseline, chosen for its bidirectional self-attention mechanism. It processes each word in the context of the whole sentence, capturing meanings that classical models miss.
- **RoBERTa + SCL (Supervised Contrastive Learning):** Our core architectural modification. We improve RoBERTa's fine-tuning phase by adding an SCL loss function. This explicitly pulls reviews with the same sentiment closer together in the vector space, while pushing opposite classes apart — creating clearer boundaries and improving accuracy on confusing texts.

3.1 Dataset

In the literature on binary sentiment analysis, SST-2 and IMDB are widely used as benchmark datasets due to their complementary characteristics. SST-2 contains short, sentence-level texts where sentiment is expressed with limited context, while IMDB consists of longer, review-based texts that provide richer contextual information and often clearer sentiment signals. Together, they allow evaluation on texts of different lengths and levels of complexity.

3.1.1 SST-2 (Stanford Sentiment Treebank)

General Description SST-2 is derived from the Stanford Sentiment Treebank (Socher et al., 2013), which consists of movie review snippets collected from Rotten Tomatoes. The SST-2 version retains sentence-level instances for sentiment analysis. As a result, the dataset contains short, self-contained sentences in which sentiment must often be inferred from limited context.

Annotation The original Stanford Sentiment Treebank was annotated manually at the phrase and sentence level using fine-grained sentiment categories. In the SST-2 version, these annotations are reduced to two sentiment classes. In SST-2, label 0 corresponds to negative sentiment, while label 1 corresponds to positive sentiment.

Data Distribution We use the standard SST-2 train, validation, and test splits provided by GLUE. The train split contains 67,349 instances, and the validation split contains 872 instances. In the train split, 29,780 samples are labeled as negative and 37,569 as positive. In the validation split, 428 samples belong to label 0 and 444 to label 1. This indicates a slight predominance of the positive class, while the overall distribution remains close to balanced.

The test split does not include the true labels. Therefore, test performance cannot be evaluated locally. Official test results are obtained by uploading the predictions on the test set to the GLUE evaluation server.

Relevance for the Model The word and character count statistics reveal clear differences across the SST-2 splits. The training split contains significantly shorter sentences, with a mean length of 9.4 words (median 7) and 53.5 characters (median 39). In contrast, both the validation and test splits exhibit substantially longer inputs, with mean sentence lengths of approximately 19 words and over 100 characters on average. Overall, this indicates that the model is trained primarily on shorter inputs with limited contextual information. Consequently, performance on the longer validation and test examples may be weaker, as these examples require richer and more complex context than is typically present in the training data.

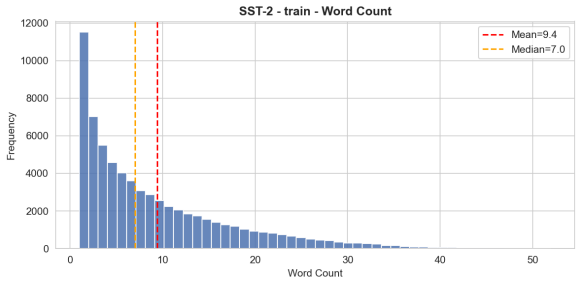

Figure 1: Word count distribution for the SST-2 training split. The mean word count is 9.4 and the median is 7.

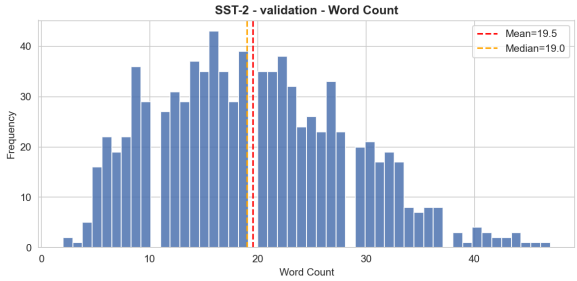

Figure 2: Word count distribution for the SST-2 validation split. The mean word count is 19.5 and the median is 19.

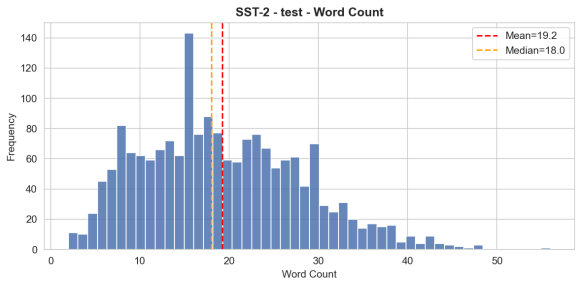

Figure 3: Word count distribution for the SST-2 test split. The mean word count is 19.2 and the median is 18.

3.1.2 IMDB (Large Movie Review Dataset)

General Description IMDB is based on the Large Movie Review Dataset (Maas et al., 2011),

which consists of full-length movie reviews collected from the Internet Movie Database. The dataset contains document-level instances for sentiment analysis, where each review is labeled as either positive or negative. Due to the length and richness of the reviews, each instance provides substantially more contextual information, allowing models to better capture sentiment cues and make more informed classification decisions.

Annotation Annotations in the IMDB dataset are provided at the document level, where each review is labeled based on its overall sentiment. The labels are derived from user-provided ratings, with reviews rated 1–4 considered negative and those rated 7–10 considered positive, while neutral reviews are excluded. As a result, the dataset is clearly polarized, containing only two sentiment classes. In IMDB, label 0 corresponds to negative sentiment, while label 1 corresponds to positive sentiment.

Data Distribution The IMDB dataset consists of 25,000 training instances and 25,000 test instances, with an equal distribution of 12,500 negative and 12,500 positive samples in each split.

A validation set is obtained by splitting 20% of the training data, resulting in 20,000 training instances and 5,000 validation instances. The split is performed in a stratified manner using a fixed random seed of 42, ensuring an equal number of samples from each class: 10,000 negative and 10,000 positive in the training set, and 2,500 negative and 2,500 positive in the validation set.

Relevance for the Model The word count distributions across the train, validation, and test splits are highly consistent, with mean values around 230 words and medians around 174 words. Most reviews fall within a moderate length range, while a smaller proportion are significantly longer.

This consistency means that the model sees similar input lengths during both training and evaluation, which reduces the risk of distribution shift. In addition, the relatively long reviews provide enough context for the model to capture sentiment across multiple sentences rather than relying only on isolated words.

3.1.3 Cross-Dataset Exploratory Analysis

Top Words per Label Across both SST-2 (train and validation splits) and IMDB (all splits), the most frequent words for both labels are largely iden-

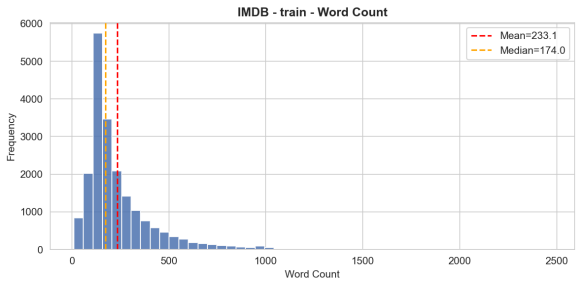

Figure 4: Word count distribution for the IMDB training split. The mean word count is 233.1 and the median is 174.

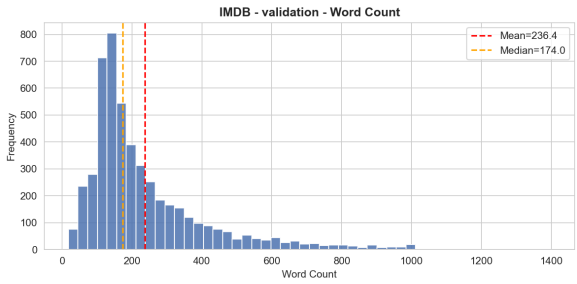

Figure 5: Word count distribution for the IMDB validation split. The mean word count is 236.4 and the median is 174.

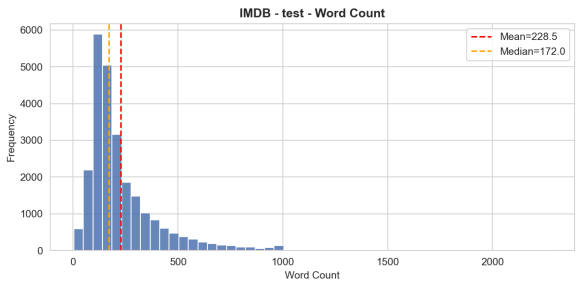

Figure 6: Word count distribution for the IMDB test split. The mean word count is 228.5 and the median is 172.

tical and dominated by common function words and punctuation such as “the”, “a”, “and”, “of”, “to”, and “.”, along with formatting tokens like “<br />” in IMDB. This shows that unigram frequency is dominated by general language patterns and heavily influenced by general language structure rather than sentiment.

Still, some label-specific differences appear in both datasets. In SST-2, negative samples contain more negation-related words such as “not”, “too”, and “n’t”, while positive samples include more explicitly favorable words such as “good” and “funny”. Similarly, in IMDB, negative reviews contain slightly more negation-related words such as “not” and contrastive terms like “but”, while positive reviews include more explicitly evaluative words such as “good”, “like”, and “very”. However, these differences are relatively small compared to the overall similarity between the two distributions.

These patterns suggest that single-word distributions provide only limited separation and that individual words alone are not enough for reliable sentiment classification, while also showing that negative sentiment is often expressed through negation and positive sentiment through directly evaluative words, requiring the model to capture broader context.

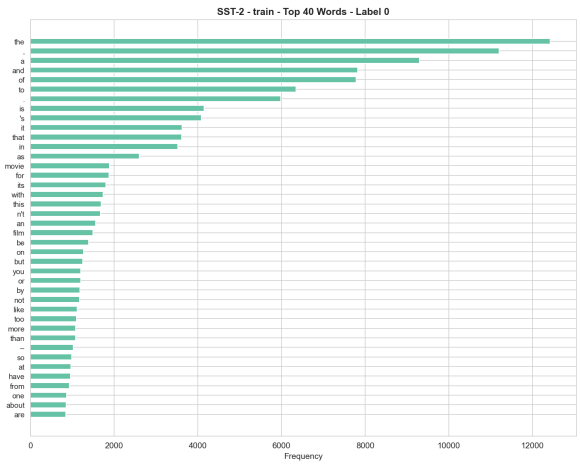

Figure 7: Most frequent words for label 0 in the SST-2 training split. Negative samples show frequent function words together with negation-related forms.

N-grams A similar pattern can be observed for both SST-2 (train and validation splits) and IMDB (all splits), where the most frequent n-grams are dominated by generic constructions such as “of the”, “in the”, “the film”, “this movie”, and “the movie is”, reflecting common review language and narrative patterns rather than sentiment. As a result, these frequent surface patterns provide limited information for distinguishing between positive and negative samples.

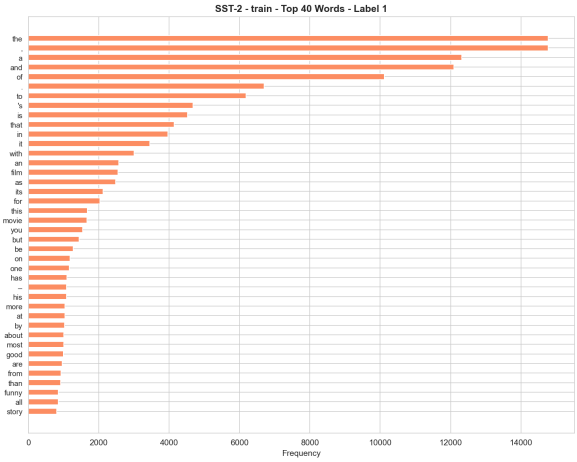

Figure 8: Most frequent words for label 1 in the SST-2 training split. Positive samples combine common high-frequency tokens with more explicit sentiment-related words.

However, trigrams capture clearer sentiment cues in both datasets. In SST-2, positive samples (label 1) include strongly evaluative expressions such as “one of the best”, “of the best”, “a good time”, and “the best of”, while negative samples (label 0) are characterized by negation and critical structures such as “it ’s not”, “does n’t have”, “there ’s no”, and “of the worst”. Similarly, in IMDB, positive samples include explicitly favorable patterns such as “one of my” and “of the best”, while negative samples are characterized by more critical and contrastive expressions such as “of the worst”, “could have been”, and “would have been”.

Taken together, the results suggest that trigrams capture more informative local context, making them more useful for sentiment classification than the most frequent surface patterns. This highlights the importance of modeling compositional semantics rather than relying on shallow bag-of-words representations.

Stopword Ratio For both datasets, the stopword ratio is consistently centered around similar values across all splits, with stable distributions indicating that a substantial portion of each sentence/review is composed of common functional words. In SST-2, the average is approximately 0.31 in train and around 0.36 in both validation and test, with most samples falling within the 0.2–0.5 range. In

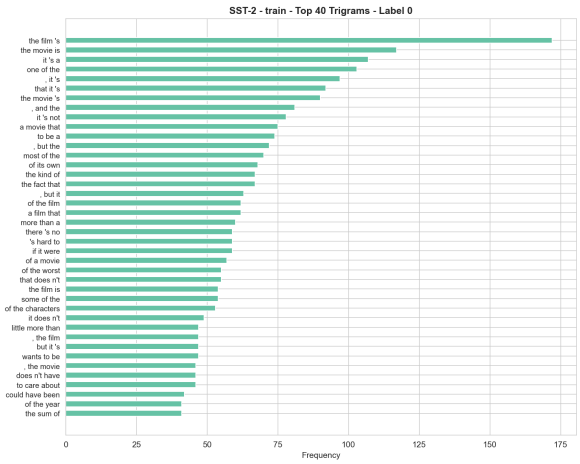

Figure 9: Most frequent trigrams for label 0 in the SST-2 training split. Negative samples are characterized by recurring negation-heavy and critical phrase patterns.

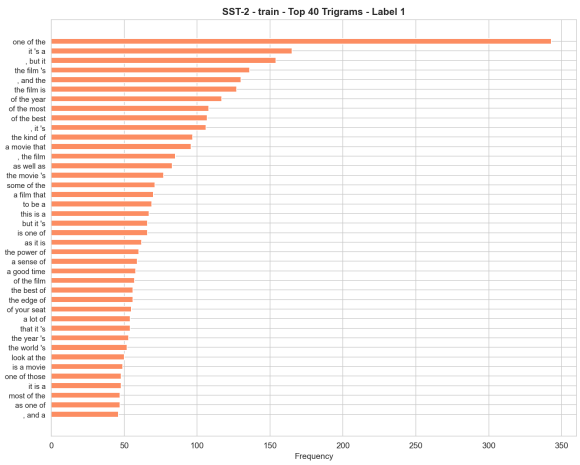

Figure 10: Most frequent trigrams for label 1 in the SST-2 training split. Positive samples include more explicitly evaluative phrase patterns.

contrast, IMDB shows higher values overall, with means close to 0.45 and most samples falling between 0.35 and 0.55. The validation split in IMDB shows a slightly wider spread toward lower values, while the test split is marginally more concentrated.

Despite these differences in absolute values, the distributions remain consistent across splits in both datasets, confirming dataset stability. Overall, this suggests that sentiment cannot be identified reliably from stopword usage or isolated keywords alone, and that contextual patterns remain important.

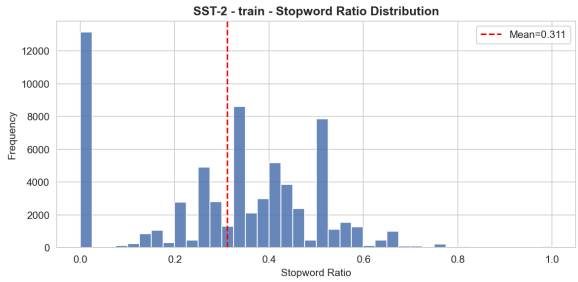

Figure 11: Stopword ratio distribution in the SST-2 training split. Most samples are concentrated in the mid-range, indicating a substantial presence of functional words.

Punctuation Distribution The punctuation distribution in both SST-2 and IMDB is dominated by a small set of characters, primarily commas and periods, followed by quotation marks, while apostrophes also appear frequently due to contractions. In SST-2, commas are the most frequent character by a large margin, with approximately 26k occurrences in train, 1k in validation, and 2.1k in test, followed by periods and quotation marks. In IMDB, periods and commas dominate similarly, with additional presence of slashes among the more frequent symbols.

In contrast, punctuation such as question marks and exclamation points occurs only marginally in both datasets. This indicates that a limited set of punctuation marks dominates the corpora and that punctuation mostly reflects sentence structure rather than contributing meaningful information for sentiment classification.

3.2 Preprocessing

Initially, before applying our custom preprocessing pipeline, we evaluated the cleaned IMDb review text with a classical SVM classifier using both Bag of Words (BoW) and TF-IDF, in order to decide which representation should be used in the follow-

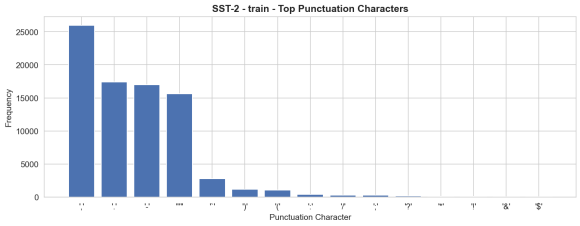

Figure 12: Punctuation character distribution in the SST-2 training split. A small set of punctuation marks dominates the corpus, primarily reflecting sentence structure.

ing experiments.

TF-IDF performed better than BoW, indicating that it provides a more discriminative representation of the reviews. On the validation set, TF-IDF achieved an accuracy of 0.9148, compared to 0.9016 for BoW, while on the test set it reached 0.8993, compared to 0.8876. The same trend is reflected by AUC, where TF-IDF obtained 0.970 on validation and 0.964 on test, whereas BoW reached 0.9593 and 0.9532. TF-IDF also produced fewer test errors, with 2518 misclassified reviews compared to 2811 for BoW. These results show that TF-IDF separates positive and negative reviews more effectively and generalizes better on unseen data. The coefficient analysis supports this conclusion. In the BoW-based SVM, many of the highest-weighted features are common sentiment words such as *bad*, *great*, *good*, *love*, and *boring*, whose importance is strongly influenced by raw frequency. In contrast, the TF-IDF-based SVM assigns higher weight to more specific and informative features, including expressions such as *excellent*, *superb*, *must see*, *well worth*, *love this*, and *not worth*. Since both models used an n-gram range of 1 to 3, this suggests that TF-IDF makes better use of informative multi-word expressions, while reducing the impact of overly common terms. Overall, TF-IDF yields a cleaner feature space and was therefore selected for the classical methods. We applied a light normalization pipeline to reduce superficial variation in the IMDb reviews. This included lowercasing, removing simple formatting artifacts such as HTML break tags, quotation marks, and special whitespace characters, reconstructing spaced contractions such as *i 'm* or *ca n 't*, expanding common contractions such as *don't* to *do not*, handling a few irregular forms, simplifying remaining *'s* cases, and performing limited punctuation normalization followed by whitespace cleanup. The goal was to make the vocabulary more consistent while only removing obvious noise from the text. This preprocessing led to only a marginal improvement in performance. Using TF-IDF with SVM, the test accuracy increased from 0.8993 on the original reviews to 0.9011 after preprocessing, which shows that the gain was very small. We then also experimented with removing the most frequent punctuation marks, namely the period and the comma, but this did not lead to a substantial improvement either, as the accuracy reached only 0.9039. In addition, removing stopwords caused a clear drop in performance, with the test accuracy decreasing to 0.8812. Based on these results, we decided to keep the initial preprocessing strategy, which focuses only on removing noise without applying more aggressive text simplifications that could discard useful sentiment information.

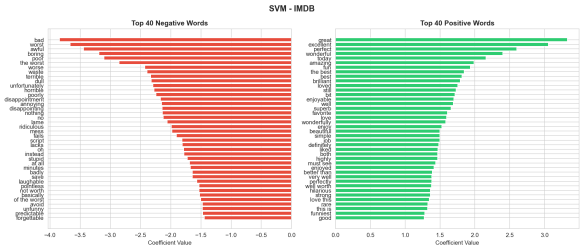

Figure 13: Top weighted negative and positive features learned by the SVM model using the TF-IDF representation on the IMDb dataset. Compared to BoW, TF-IDF highlights more specific and more discriminative words and n-grams.

4 Models

4.1 Classical and Hybrid Models

4.1.1 Support Vector Machine (SVM)

#	C	N-gram	MaxFeat	SubTF	Val Acc
1	0.5	(1,3)	200000	False	0.8326
2	0.5	(1,3)	300000	False	0.8326
3	1	(1,3)	200000	True	0.8314
4	1	(1,3)	300000	True	0.8314
5	0.5	(1,3)	200000	True	0.8303
6	1	(1,3)	200000	False	0.8303
7	1	(1,2)	300000	True	0.8303
8	0.5	(1,2)	300000	True	0.8291
9	0.1	(1,2)	200000	True	0.8131
10	0.1	(1,3)	300000	False	0.8073

Table 1: Selected SVM configurations from grid search on the SST-2 dataset

#	C	N-gram	MaxFeat	SubTF	Val Acc
1	1	(1,3)	200000	False	0.9128
2	1	(1,3)	300000	False	0.9114
3	0.5	(1,3)	200000	False	0.9108
4	1	(1,2)	200000	True	0.9102
5	0.5	(1,3)	300000	False	0.9098
6	0.5	(1,3)	300000	True	0.9094
7	0.5	(1,2)	200000	False	0.9084
8	0.1	(1,2)	200000	True	0.8994
9	0.1	(1,3)	200000	False	0.8972
10	0.1	(1,3)	300000	False	0.8944

Table 2: Selected SVM configurations from grid search on the IMDB dataset

The training protocol consists of first fixing a sufficiently large number of iterations (200) to ensure that the SVM model converges. Based on this setting, a grid search was performed over multiple configurations of the SVM model and TF-IDF vectorizer.

On the SST-2 dataset, the best configuration obtained a validation accuracy of 0.8326 and a validation loss of 0.4895. On the test set, the model achieved an accuracy of 0.837. These results, among other top configurations, can be seen in Table 1.

The same approach was applied to the IMDB dataset, where the best configuration achieved a validation accuracy of 0.9128 and a validation loss of 0.3056. On the test set, the selected model achieved an accuracy of 0.9011 and a loss of 0.334. These results, among other top configurations, can be seen in Table 2.

4.1.2 Logistic Regression

#	C	N-gram	MaxFeat	Solver	Val Acc
1	10.0	(1,2)	50000	lbfgs	0.8291
2	1.0	(1,1)	50000	lbfgs	0.8257
3	1.0	(1,1)	100000	lbfgs	0.8257
4	1.0	(1,1)	200000	lbfgs	0.8257
5	10.0	(1,2)	50000	liblinear	0.8257
6	10.0	(1,2)	100000	lbfgs	0.8257
7	10.0	(1,2)	200000	lbfgs	0.8257
8	10.0	(1,3)	200000	liblinear	0.8257
9	5.0	(1,2)	100000	liblinear	0.8245
10	5.0	(1,2)	200000	liblinear	0.8245

Table 3: Selected Logistic Regression configurations from grid search on the SST-2 dataset

On the SST-2 dataset, the best Logistic Regression configuration obtained a validation accuracy of 0.8291 and a validation loss of 0.4812. On the test set, the model achieved an accuracy of 0.834. These results, among other top configurations, can be seen in Table 3.

#	C	N-gram	MaxFeat	Solver	Val Acc
1	10.0	(1,2)	50000	lbfgs	0.9118
2	10.0	(1,2)	100000	lbfgs	0.9116
3	10.0	(1,2)	50000	liblinear	0.9108
4	10.0	(1,3)	200000	lbfgs	0.9106
5	10.0	(1,2)	100000	liblinear	0.9104
6	10.0	(1,3)	50000	liblinear	0.9098
7	5.0	(1,2)	100000	lbfgs	0.9094
8	10.0	(1,3)	50000	lbfgs	0.9094
9	5.0	(1,2)	200000	lbfgs	0.9092
10	5.0	(1,2)	50000	lbfgs	0.9090

Table 4: Selected Logistic Regression configurations from grid search on the IMDB dataset

The same approach was applied to the IMDB dataset, where the best configuration achieved a validation accuracy of 0.9118 and a validation loss of 0.2623. On the test set, the selected model achieved an accuracy of 0.9092 and a test loss of 0.2832. These results, among other top configurations, can be seen in Table 4.

4.1.3 Naive Bayes + Support Vector Machine (NB-SVM)

#	C	N-gram	MaxFeat	SubTF	Val Acc
1	0.10	(1,3)	100000	False	0.8406
2	0.05	(1,2)	50000	False	0.8383
3	0.10	(1,2)	50000	True	0.8383
4	0.10	(1,2)	50000	False	0.8383
5	0.10	(1,3)	100000	True	0.8383
6	0.05	(1,2)	50000	True	0.8372
7	0.05	(1,3)	100000	False	0.8372
8	0.10	(1,3)	50000	True	0.8372
9	0.05	(1,3)	100000	True	0.8360
10	0.10	(1,3)	50000	False	0.8360

Table 5: Selected NB-SVM configurations from grid search on the SST-2 dataset

On the SST-2 dataset, the best Naive Bayes configuration obtained a validation accuracy of 0.8406 and a validation loss of 0.4944. On the test set, the model achieved an accuracy of 0.847. These results, among other top configurations, can be seen in Table 5

The same approach was applied to the IMDB dataset, where the best configuration achieved a validation accuracy of 0.9152 and a validation loss of 0.2769. On the test set, the selected model

#	C	N-gram	MaxFeat	SubTF	Val Acc
1	5.0	(1,3)	200000	True	0.9162
2	5.0	(1,2)	200000	False	0.9158
3	5.0	(1,2)	300000	False	0.9158
4	5.0	(1,3)	300000	True	0.9156
5	10.0	(1,3)	200000	False	0.9156
6	10.0	(1,3)	300000	False	0.9156
7	5.0	(1,3)	200000	False	0.9154
8	2.0	(1,3)	200000	True	0.9152
9	2.0	(1,3)	300000	True	0.9148
10	5.0	(1,3)	300000	False	0.9148

Table 6: Selected NB-SVM configurations from grid search on the IMDB dataset

achieved an accuracy of 0.908 and a test loss of 0.2992. These results, among other top configurations, can be seen in Table 6

4.1.4 Limitations

Limitation: Inability to handle contrast and polarity shifts.

For example, in the sentence **"Comanche Moon had everything going for it… high-quality TV westerns. Unfortunately, the problems with 'Moon'…"**, the model is influenced by positive expressions such as "everything going for it" and "high-quality", while failing to properly account for the negative conclusion introduced by "Unfortunately".

Similarly, in **"This was going to be another made-for-video crap fest… Instead, I found…"**, the model is misled by the negative phrase "crap fest", even though the final sentiment becomes positive due to the contrast introduced by "Instead".

Limitation: Inability to understand irony and mixed sentiment.

In the sentence **"That's a bad, raunchy, predictable, tacky, salacious soap opera… 'The Best of Everything' is just such a guilty pleasure for me"**, the model focuses on strongly negative words such as "bad", "tacky", and "raunchy", while ignoring the fact that "guilty pleasure" expresses a positive overall opinion.

Limitation: Poor aggregation of sentence-level sentiment. For example, in **"To summarize this movie: nice for TV but too small for the theatre. I enjoyed watching this movie at home but I wont watch it a second time. The concept is good, but ..."**, the model encounters both positive expressions such as "nice", "enjoyed", and "good", and negative cues such as "too small for the theatre" and "wont watch it a second time". This shows that SVM does not properly aggregate sentiment across the entire sentence and may be influenced too strongly by local positive words, even when the overall evaluation is negative.

Limitation: Confusion between description and opinion.

In the sentence **"A depressed creepy teenager does many bad things…"**, the model interprets words like "creepy" and "bad" as indicators of negative sentiment, although they are simply describing the plot rather than evaluating the film.

Limitation: Difficulty handling negation and complex structures.

For example, in **"I had low expectations on the movie, and still they didn't come up them"**, the logical meaning of the sentence is not properly captured, as SVM does not model relationships between words, but only their individual presence.

Limitation: Over-reliance on isolated sentiment words.

In **"I thought this movie was really good…"**, the phrase "really good" dominates the prediction, even though the overall review may later contain negative elements.

In conclusion, these examples demonstrate that SVM, even when combined with TF-IDF representations, suffers from several key limitations, including **inability to capture context**, **difficulty handling irony**, **poor understanding of negation**, and **lack of modeling semantic relationships between words**.

These issues arise because SVM relies primarily on the **presence and importance of individual words**, as encoded by TF-IDF weights, rather than understanding how words interact within a sentence. As a result, the model treats features independently and cannot capture dependencies, word order, or compositional meaning. This leads to incorrect predictions in cases where the sentiment depends on context, contrast, or relationships between words, especially in complex, mixed, or ambiguous sentences.

4.2 Deep Learning: Long Short-Term Memory (LSTM)

4.2.1 Theoretical Background

While our classical baselines (SVM and Logistic Regression) performed surprisingly well, our earlier error analysis revealed a fundamental limitation: they rely entirely on frequency-based feature representations like TF-IDF. By treating words as

independent features, these models fail to capture word dependencies, word order, and compositional meaning.

To address this, we transitioned to Deep Learning, specifically Long Short-Term Memory (LSTM) networks. Unlike classical models, LSTMs are a type of Recurrent Neural Network (RNN) designed to process data sequentially. They maintain a hidden state that is updated as the model reads a text word by word, allowing them to theoretically aggregate sentiment across an entire sentence. Through their internal gating mechanisms, LSTMs mitigate the vanishing gradient problem common in standard RNNs, allowing them to "remember" earlier parts of a sequence. In theory, this architectural shift represents a major step forward, as it should allow the model to natively understand context, contrastive shifts, and negation without relying on engineered n-grams.

4.2.2 Training and Hyperparameter Tuning

To ensure a fair comparison against our highly optimized classical models, we trained the LSTM architectures using word embeddings to represent our vocabulary. We employed a grid search methodology to identify the best hyperparameter configurations for both datasets. We iterated over key parameters, including the hidden state dimensionality, learning rate, dropout rates (to prevent overfitting), and the number of LSTM layers. After isolating the optimal configurations based on validation performance, we evaluated the best models on the respective test sets.

4.2.3 Results and Discussion

The results of the LSTM models presented an interesting contrast when compared to the classical baselines.

On the **SST-2 dataset**, the LSTM achieved an accuracy of approximately 85-86%, representing a noticeable improvement over the 83% achieved by the classic models. This gain aligns with our theoretical expectations. Because **SST-2** consists of short sentences, with a median of 18 words in the test set, the LSTM can easily process the entire sequence without losing context. This allows it to successfully capture local syntactic nuances, such as short-range negations and compositional phrases, that the bag-of-words approach missed.

However, on the **IMDB dataset**, the LSTM achieved an accuracy of 89-90%, showing no improvement over the classical models. While this may initially seem counterintuitive, it is highly realistic given the dataset's characteristics. The IMDB test set consists of long document-level reviews with a mean length of roughly 228 words. Despite their gating mechanisms, LSTMs still struggle with extremely long-range dependencies; the sentiment signal can become diluted over hundreds of tokens. Furthermore, our exploratory analysis demonstrated that IMDB contains strong, highly discriminative lexical signals (e.g., "superb", "not worth"). The SVM perfectly captures these high-frequency global features, making a complex sequential model unnecessary if the sentiment is heavily keyword-driven.

Ultimately, these results highlight that while sequential modeling (LSTM) offers a slight benefit for short-form text, it suffers from diminishing returns on long-form text. To truly break the 90% ceiling on complex documents, the architecture must evolve beyond simple recurrence to mechanisms that can directly attend to all parts of a sequence simultaneously, setting the stage for Transformer-based architectures.

4.3 RoBERTa and RoBERTa + SCL

RoBERTa+SCL represents the strongest model in our comparative study because it addresses several limitations that appeared in the classical, hybrid and LSTM-based approaches. Classical models such as SVM, Logistic Regression and Naive Bayes mainly rely on the presence and weight of individual words. This makes them effective when the sentiment is expressed directly, through words such as “good”, “bad”, “superb” or “not worth”, but weaker when the real meaning depends on context, contrast, negation, irony or the relation between multiple parts of the review. In these cases, the model may classify a review incorrectly because it reacts too strongly to visible sentiment words and does not understand how those words function inside the sentence. LSTM improves this problem partially by reading the text as a sequence, but its advantage is mostly visible on short inputs. In longer reviews, the relevant opinion can be distributed across several sentences, and the sentiment signal may become harder to preserve until the final prediction. RoBERTa addresses part of these issues through its tokenizer and attention layers. The tokenizer converts the review into smaller units that the model can process, while the attention mechanism allows the model to compare words and phrases

with each other inside the same review. As a result, RoBERTa does not only detect that a word such as “good”, “bad”, “crap” or “high-quality” appears, but can also analyse whether that word is contradicted, reinforced or reinterpreted by the surrounding context.

This mechanism explains why RoBERTa can solve part of the errors observed in the SVM-based analysis. One important problem was the incorrect handling of contrast. For example, in “Comanche Moon had everything going for it... high-quality TV westerns. Unfortunately, the problems with ‘Moon’...”, the classical model is influenced by the positive expressions at the beginning and does not properly account for the negative conclusion introduced by “Unfortunately”. RoBERTa is better suited for this case because the attention layers allow the model to connect the initial praise with the later contrastive marker and the negative conclusion. A similar situation appears in “This was going to be another made-for-video crap fest... Instead, I found...”, where the word “Instead” changes the direction of the review. A model based mostly on word weights can be dominated by “crap fest”, while RoBERTa can use the later context to understand that the final opinion may become positive. Therefore, RoBERTa partially reduces the over-reliance on isolated sentiment words by modelling how the sentence develops.

Another limitation addressed by RoBERTa is the confusion between local sentiment words and the actual global opinion. In mixed or ironic reviews, negative words do not always imply a negative review. For example, in “That’s a bad, raunchy, predictable, tacky, salacious soap opera... ‘The Best of Everything’ is just such a guilty pleasure for me”, words such as “bad”, “raunchy” and “tacky” are strongly negative if they are considered separately, but the phrase “guilty pleasure” changes the interpretation of the whole fragment. RoBERTa can better capture this type of dependency because the attention mechanism allows the final positive framing to influence how the previous negative words are interpreted. The same idea applies to plot descriptions such as “A depressed creepy teenager does many bad things...”. In this case, words like “creepy” and “bad” may describe the character or the events of the film, not the reviewer’s evaluation of the movie. Since RoBERTa analyses words in relation to the rest of the sentence, it is more capable of distinguishing between description and opinion than models that treat these words mainly as independent negative indicators.

RoBERTa also improves the aggregation of sentiment across the review. In examples where the text contains both positive and negative signals, such as “nice”, “enjoyed” and “good” together with “too small for the theatre” and “wont watch it a second time”, the final sentiment cannot be inferred from one phrase alone. The model must decide which parts express the final evaluation and how the different statements interact. Classical models often fail here because they sum or weight word-level evidence without truly modelling the logical relation between statements. LSTM can capture some of this structure in shorter sentences, but in long reviews it may still lose important information because it processes the text sequentially. RoBERTa is better suited for this situation because attention allows distant parts of the review to influence each other more directly. This does not mean that RoBERTa eliminates all ambiguity, but it gives the model a stronger mechanism for combining sentence-level cues into a more coherent document-level prediction.

The SCL component adds a different type of improvement. RoBERTa improves how the review is read, while SCL improves how the model learns to organise the reviews during training. With standard cross-entropy, the model mainly learns to assign the correct label to each example. However, two reviews from different classes may still end up with similar internal representations if they contain similar words. Supervised contrastive learning adds an extra constraint: reviews with the same sentiment label are pulled closer together in the representation space, while reviews with different labels are pushed farther apart. In practical terms, SCL helps the model build a cleaner internal separation between positive and negative reviews. This is useful for cases where the surface vocabulary is misleading. For example, a positive review that contains negative words such as “bad”, “tacky” or “guilty pleasure” should still be represented closer to other positive reviews, while a negative review that contains positive expressions such as “nice”, “good” or “high-quality” should be represented closer to other negative reviews if its overall judgement is negative.

Therefore, the contribution of SCL is not that it directly understands contrast, negation or irony by itself. That contextual understanding mainly

comes from RoBERTa's attention layers. SCL improves the quality of the representations produced by RoBERTa by making examples from the same class more compact and examples from different classes more separated. This makes the classifier less unstable in ambiguous cases, because the decision is supported not only by the final classification layer, but also by a representation space where sentiment classes are more clearly structured. As a result, RoBERTa+SCL is better equipped for reviews with mixed signals, implicit sentiment, contrastive structures and misleading local vocabulary.

Overall, RoBERTa+SCL performs better because it targets the main weaknesses identified in the earlier stages of the study without relying on a single mechanism. RoBERTa reduces part of the errors caused by isolated word interpretation, weak handling of contrast, poor treatment of negation, confusion between plot description and opinion, and limited modelling of long-range relationships. SCL then strengthens the model by making the learned positive and negative representations more consistent and easier to separate. For this reason, the improvement of RoBERTa+SCL is not only a numerical gain, but also a qualitative one: it shows that a state-of-the-art approach is better aligned with the real structure of sentiment in movie reviews, where the final label often depends on context, argumentation and the interaction between multiple parts of the text.

Results summary. On SST-2, RoBERTa-CE reaches 93.69% validation accuracy and RoBERTa-SCL reaches 94.38%. A 3×3 grid over temperature and λ shows that every SCL configuration exceeds the CE baseline, with accuracies ranging from 94.15% to 94.61%. On IMDB, RoBERTa-CE reaches 93.82% test accuracy and RoBERTa-SCL reaches 93.90%. RoBERTa+SCL also reaches 93.90% test accuracy against the SVM's 90.11%, a gap of $+3.79$ percentage points, which means 949 fewer misclassifications across the $25{,}000$-review test set.

$\tau \setminus \lambda$	0.1	0.5	1.0
0.1	94.27	**94.61**	94.15
0.3	94.50	94.27	94.27
0.5	94.38	94.15	94.15

Table 7: SST-2 validation accuracy (%) for RoBERTa-SCL across the 3×3 grid of temperature τ and λ. All configurations exceed the CE baseline (93.69%). Best cell in bold.

5 Future Work

While our benchmarking study provided a robust overview of the evolution of binary sentiment analysis architectures, a logical next step would be moving beyond simple positive/negative classifications to more nuanced sentiment modelling. The original Stanford Sentiment Treebank actually contains fine-grained sentiment categories, which were reduced to just two classes for the SST-2 version. Expanding our models to predict a wider spectrum of emotions, such as neutral, highly positive, or mixed sentiments, would allow us to capture the true complexity of human language. This is particularly relevant for the IMDB dataset, where our error analysis showed reviews often contain conflicting opinions (e.g., praising the acting while criticising the plot).

6 Conclusion

In this project, we went through the general concepts of Natural Language Processing needed to build a binary sentiment analysis system. We worked with text cleaning, tokenization and preprocessing, then transformed text into numerical representations using Bag-of-Words, n-grams, TF-IDF and GloVe embeddings. This helped us understand how raw text can be prepared and used by machine learning models.

We also worked with several types of models, starting from classical methods such as SVM, Logistic Regression, Naive Bayes and the hybrid Naive Bayes + SVM approach, then moving to more advanced models such as LSTM, BERT and RoBERTa. This progression helped us understand how sentiment analysis evolved from simple feature-based methods to models that use context more effectively.

Overall, we liked the project because it allowed us to apply the general concepts of Natural Language Processing learned in this course, such as preprocessing, text representation, model training and evaluation. There was no specific part that we disliked. The most challenging part was understanding the differences between the models and keeping the comparison consistent across IMDB and SST-2.

6.1 Limitations

One limitation of our work is related to computational resources. The classical and hybrid models, such as SVM, Logistic Regression, Naive Bayes

and the hybrid Naive Bayes + SVM approach, were lightweight enough to be trained and tested on our own laptops. These experiments can run on a normal laptop configuration, for example a recent Intel Core i5/i7 or AMD Ryzen 5/7 processor, 16 GB of RAM and no dedicated GPU. Since these models use simpler text representations such as Bag-of-Words, n-grams and TF-IDF, they require less memory and shorter training times.

The Transformer-based models were more demanding. For both BERT and RoBERTa, we rented an NVIDIA RTX 5090 GPU with 32 GB of memory, because fine-tuning these models requires more memory and longer training times, especially on a larger dataset such as IMDB. The RoBERTa model combined with supervised contrastive learning was even more expensive to train, because it had to optimize both the normal classification objective and the supervised contrastive objective.

Another limitation is that our study focuses only on English binary sentiment analysis, using IMDB and SST-2. Therefore, the results do not show how well the same models would work on other languages, multilingual datasets or sentiment tasks with more than two classes. Also, the comparison depends on the selected hyperparameters, available training time and hardware resources. Future work could test the models on more datasets, include other languages and explore more efficient versions of Transformer-based models.

7 Ethical Statement

A broader ethical risk of sentiment analysis is that it can be used to build profiles of people based on the texts they write online. If a system classifies messages, reviews, comments or posts as positive, negative, aggressive, suspicious or emotionally unstable, these predictions can be stored and combined over time. This can create a behavioral profile of a person, even if the person did not explicitly agree to being analyzed in this way.

This becomes more serious if such systems are used by state institutions. Automatic sentiment classification could be used to monitor citizens, detect criticism, identify people considered problematic, or evaluate public reactions to political and social events. In this context, the issue is not only technical performance, but also the possibility of surveillance and loss of privacy. People should have the right to express opinions without being permanently profiled by automated systems.

Wrong classification can also have serious consequences. A sarcastic comment, a joke, an angry message written in a specific context, or a misunderstood opinion could be classified as dangerous, hostile or suspicious. If such a prediction is used in legal, administrative or security-related decisions, an innocent person could be unfairly investigated, flagged or treated as a risk. This shows why sentiment analysis should not be used as direct evidence or as the only basis for decisions that affect people.

Another important concern is the right to anonymity. Online opinions can reveal personal preferences, political views, emotional states and social attitudes. If sentiment analysis is combined with identity tracking, it becomes possible to connect anonymous texts to a long-term profile. For this reason, systems that analyze sentiment at scale should respect privacy, avoid unnecessary data storage, and include strong limits on how predictions are used.

In our project, sentiment analysis is used only for academic comparison on IMDB and SST-2. However, in real-world applications, such systems should be transparent, limited in scope and supervised by humans. Their outputs should be treated as uncertain predictions, not as facts about a person.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.

Beliz Gunel, Jingfei Du, Alexis Conneau, and Veselin Stoyanov. 2021. Supervised contrastive learning for pre-trained language model fine-tuning. In *International Conference on Learning Representations (ICLR 2021)*.

Sepp Hochreiter and Jurgen Schmidhuber. 1997. Long short-term memory. *Neural Computation*, 9(8):1735–1780.

Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *European Conference on Machine Learning*, pages 137–142. Springer.

Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. Supervised contrastive learning. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.

Yoon Kim. 2014. Convolutional neural networks for sentence classification. In *Proceedings of the 2014*

Conference on Empirical Methods in Natural Language Processing, pages 1746–1751.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692*.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, Portland, Oregon, USA. Association for Computational Linguistics.

Andrew McCallum and Kamal Nigam. 1998. A comparison of event models for naive bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*, pages 41–48.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, pages 3111–3119.

Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. 2002. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 79–86.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1532–1543.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642, Seattle, Washington, USA. Association for Computational Linguistics.

Peter D. Turney. 2002. Thumbs up or thumbs down? semantic orientation applied to unsupervised classification of reviews. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 417–424.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need.

Sida Wang and Christopher D. Manning. 2012. Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 90–94.

Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alex Smola, and Eduard Hovy. 2016. Hierarchical attention networks for document classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 1480–1489.